

ShopEz

Full Stack MERN E-Commerce Application

Project Technical Documentation

Developed by: SIDDANI SRI CHARAN

Email: charaniamgreat@gmail.com

Team ID: _____

Date: 18 June 2026

1. Introduction

Project Title: ShopEz

Developer: SIDDANI SRI CHARAN (Solo Project)

ShopEz is a comprehensive full-stack e-commerce solution designed to provide a seamless shopping experience. Built using the MERN stack (MongoDB, Express.js, React.js, Node.js), it bridges the gap between consumers and products through a responsive and intuitive interface.

2. Project Overview

Purpose: The goal of ShopEz is to create a robust, scalable e-commerce platform that allows users to browse cataloged products, manage a virtual shopping cart, and complete transactions securely. It also provides an administrative layer for stock and order management.

Key Features:

- **User Module:** Registration, Login (JWT), and User Profile.
- **Product Module:** Categorized browsing (Men, Women, Kids), Search, and Filtering.
- **Cart & Checkout:** Real-time cart management and Cash on Delivery (COD) processing.
- **Admin Dashboard:** Comprehensive management of product listings and order statuses.

3. System Architecture

Frontend (React.js):

Utilizes a Component-Based Architecture. State management is handled through React Context API or local state. React Router DOM manages client-side navigation.

Backend (Node.js & Express.js):

An MVC-inspired RESTful API. Express handles routing, while middleware manage authentication and data validation. The backend serves JSON responses to the frontend client.

Database (MongoDB):

A NoSQL document database. Key collections include:

- **Users:** Stores user credentials, contact info, and role (User/Admin).
- **Products:** Contains details like Name, Description, Category, Price, and Stock.
- **Orders:** Maintains order history, product references, and shipping status.

4. Setup Instructions

Prerequisites:

- Node.js (v18.x or higher)
- NPM or Yarn Package Manager
- MongoDB Community Server (Running locally on port 27017)

Installation:

1. Clone the Repository:

```
git clone https://github.com/charan/shopez.git
```

2. Backend Setup:

```
cd server && npm install
```

3. Frontend Setup:

```
cd client && npm install
```

4. Environment Variables: Create a .env file in the server directory with:

```
PORT=5000  
MONGO_URI=mongodb://localhost:27017/shopez  
JWT_SECRET=your_secret_key
```

5. Folder Structure

Directory	Description
/server/models	Mongoose schemas defining data structure for Users, Products, etc.
/server/routes	API route definitions (endpoints).

/client/src/components	Reusable UI components (Navbar, Footer, ProductCard).
/client/src/pages	Main view components (Home, ProductDetails, Checkout).

6. Running the Application

To start the application in development mode, open two terminal windows:

Start Backend:

```
cd server && npm run dev
```

Start Frontend:

```
cd client && npm start
```

7. API Documentation

Method	Endpoint	Description
POST	/api/users/login	Authenticates user and returns JWT token.
GET	/api/products	Fetches all products with optional filters.
POST	/api/orders	Creates a new COD order. (Protected)

8. Authentication & Authorization

ShopEz utilizes JSON Web Tokens (JWT) for secure state-less authentication. On login, the server generates a token signed with a secret key. This token is stored in the browser's LocalStorage or Cookies and sent in the 'Authorization' header for protected routes. Authorization is handled via custom middleware that checks user roles (e.g., verifying if a user is an Admin before granting access to /admin routes).

9. User Interface

The UI is designed with a clean, minimalist aesthetic for ease of use. It features a responsive navigation bar, category-specific grids, and a streamlined multi-step checkout process. Visual placeholders identify where media assets like product banners and high-resolution images are integrated.

10. Testing Strategy

A rigorous testing methodology was followed to ensure stability:

- **Unit Testing:** Testing individual helper functions and validation logic.
- **Integration Testing:** Verifying that the Express routes correctly interact with MongoDB.

- **Scenario:** User adds an out-of-stock item to the cart (Expected: Prevent addition and show warning).

11. Screenshots / Demo

[Link to Demo / GitHub Repository: <https://github.com/charan/shopez>]

Placeholder: Include screenshots of Home Page, Admin Panel, and Order Success screen here.

12. Known Issues

- Client-side filter reset occasionally requires a page refresh.
- Large image uploads to MongoDB may impact performance (Migration to Cloudinary planned).

13. Future Enhancements

- Integration of Online Payment Gateways (Stripe/Razorpay).
- AI-based product recommendation system.
- Progressive Web App (PWA) support for offline browsing.